

Introduction to SQL SELECT Statement

Single-Table Querying with the `classicmodels` Sample Database

DBMS Course

Smart Logistics and Supply Chain Management Lab

October 7, 2026

Learning Objectives

After this lecture, students will be able to:

- 1 Understand the purpose and role of the `SELECT` statement in SQL.
- 2 Retrieve all columns or specific columns from a table.
- 3 Use column aliases and define computed columns in query results.
- 4 Filter data using `WHERE` and comparison operators.
- 5 Sort results, limit returned rows, and eliminate duplicate values.
- 6 Write practical single-table queries on the `classicmodels` database.

What is SELECT Used For?

Core Concept

SELECT is used to retrieve data from one or more tables and present the result in a tabular format of rows and columns.

Business Questions

- Which customers are located in the USA?
- Which products have an MSRP above 150?
- Which orders have been cancelled?

Answered in SQL

```
SELECT customerName, country  
FROM customers  
WHERE country = 'USA';
```

Sample Database: classicmodels

- classicmodels models the business operations of a retailer selling scale model cars, motorcycles, planes, and ships.
- In this lecture, we focus exclusively on single-table queries.

Table	Description
products	Product catalog and inventory data
customers	Customer profiles and contact details
orders	Order headers
orderdetails	Order line items
payments	Customer payment transactions
employees	Employee directory
offices	Regional sales office locations
productlines	Product lines and categories

Setting Up the Practice Environment

Select the Database

```
USE classicmodels;
```

List All Tables

```
SHOW TABLES;
```

Inspect Table Schema

```
DESC products;  
DESC customers;  
DESC orders;
```

Best Practice

Before writing queries, always check the available columns and their data types.

Basic Syntax of SELECT

General Template

```
SELECT column1, column2, ...  
FROM table_name;
```

Clause	Meaning
SELECT	Specifies the columns to return in the result
column1, column2, ...	Comma-separated list of projected columns
FROM	Specifies the table containing the data
table_name	Target table name to query

First Query Examples

Retrieve Customer Names

```
SELECT customerName  
FROM customers;
```

Retrieve Order Numbers

```
SELECT orderNumber  
FROM orders;
```

Reading a Query

- **SELECT**: which columns?
- **FROM**: from which table?
- Semicolon (;) terminates the statement.

Retrieving All Columns with SELECT *

```
SELECT *  
FROM productlines;
```

When is it Useful?

- Quickly inspecting table data.
- Exploring table schemas while learning SQL.

Avoid in Production

On large tables, `SELECT *` can retrieve unnecessary columns, slowing down network transfer and wasting memory. In real applications, specify only required columns.

Selecting Specific Columns

Products

```
SELECT productCode, productName, productLine,  
       MSRP  
FROM products;
```

Customers

```
SELECT customerNumber, customerName, city,  
       country  
FROM customers;
```

Rule of Thumb

Project only columns necessary for the analysis or user interface.

Question

To build a customer directory grouped by country, which columns do we need?

Aliasing Columns with AS

```
SELECT
    productCode AS product_code,
    productName AS product_name,
    productLine AS product_line,
    MSRP AS suggested_retail_price
FROM products;
```

- Aliases provide clearer, user-friendly column headers in query output.
- The **AS** keyword is optional, but using it explicitly improves readability.

Computed Columns in SELECT

SELECT

```
productCode,  
productName,  
buyPrice,  
MSRP,  
MSRP - buyPrice AS profitEstimate
```

```
FROM products;
```

Key Points

- Calculations can be written directly in the **SELECT** clause.
- Computed columns only exist in the query result set.
- The underlying table schema is not altered.

```
SELECT quantityOrdered * priceEach AS lineTotal  
FROM orderdetails;
```

Filtering Data with WHERE

General Template

```
SELECT column1, column2  
FROM table_name  
WHERE condition;
```

Motorcycles Category

```
SELECT productCode, productName, productLine  
FROM products  
WHERE productLine = 'Motorcycles';
```

Customers in USA

```
SELECT customerNumber, customerName, country  
FROM customers  
WHERE country = 'USA';
```

Common Comparison Operators

Operator	Meaning	Example
=	Equal to	country = 'USA'
<>	Not equal to	status <> 'Shipped'
>	Greater than	MSRP > 150
<	Less than	quantityInStock < 1000
>=	Greater than or equal to	creditLimit >= 100000
<=	Less than or equal to	buyPrice <= 50

Remember

The condition in **WHERE** evaluates to TRUE or FALSE for each row.

Numeric Filtering Examples

MSRP Greater Than 150

```
SELECT productCode, productName, MSRP
FROM products
WHERE MSRP > 150;
```

Stock Quantity Below 1000

```
SELECT productCode, productName,
       quantityInStock
FROM products
WHERE quantityInStock < 1000;
```

```
SELECT customerNumber, customerName, creditLimit
FROM customers
WHERE creditLimit >= 100000;
```

Combining Conditions with AND

Logic

AND requires all conditions to evaluate to TRUE.

```
SELECT productCode, productName, productLine, MSRP
FROM products
WHERE productLine = 'Classic Cars'
      AND MSRP > 100;
```

```
SELECT customerNumber, customerName, country, creditLimit
FROM customers
WHERE country = 'USA'
      AND creditLimit > 100000;
```

Combining Conditions with OR

Logic

OR requires at least one condition to evaluate to TRUE.

Customers in USA or France

```
SELECT customerNumber, customerName, country
FROM customers
WHERE country = 'USA'
      OR country = 'France';
```

Motorcycles or Planes

```
SELECT productCode, productName, productLine
FROM products
WHERE productLine = 'Motorcycles'
      OR productLine = 'Planes';
```

Negating Conditions with NOT

Using NOT

```
SELECT customerNumber, customerName, country  
FROM customers  
WHERE NOT country = 'USA';
```

```
SELECT orderNumber, orderDate, status  
FROM orders  
WHERE status <> 'Shipped';
```

Equivalent Form

```
SELECT customerNumber, customerName, country  
FROM customers  
WHERE country <> 'USA';
```

Pattern Matching with LIKE

Concept

LIKE performs pattern matching on string values using wildcards.

Wildcard	Meaning
%	Matches zero or more characters
_	Matches exactly one character

```
SELECT customerName, city, country
FROM customers
WHERE customerName LIKE '%Mini%';
```

```
SELECT productCode, productName
FROM products
WHERE productName LIKE '196%';
```

Filtering by a Set of Values with IN

Concept

IN provides a clean, concise shorthand for multiple **OR** equality tests on the same column.

Using Multiple OR

```
WHERE country = 'USA'  
OR country = 'France'  
OR country = 'Germany'
```

Using IN

```
WHERE country IN ('USA', 'France', 'Germany')
```

```
SELECT customerNumber, customerName, country  
FROM customers  
WHERE country IN ('USA', 'France', 'Germany');
```

Range Filtering with BETWEEN

Concept

BETWEEN a **AND** b tests whether a value falls within the range from a to b.

```
SELECT productCode, productName, MSRP
FROM products
WHERE MSRP BETWEEN 100 AND 150;
```

```
SELECT orderNumber, orderDate, status
FROM orders
WHERE orderDate BETWEEN '2005-01-01' AND '2005-12-31';
```

Note

In SQL, **BETWEEN** is inclusive of both boundary endpoints.

Checking for NULL Values

What is NULL?

NULL represents unknown, missing, or inapplicable data. Never test for NULL using =.

Find Rows with NULL

```
SELECT customerNumber, customerName,  
       salesRepEmployeeNumber  
FROM customers  
WHERE salesRepEmployeeNumber IS NULL;
```

Find Rows Without NULL

```
SELECT customerNumber, customerName,  
       salesRepEmployeeNumber  
FROM customers  
WHERE salesRepEmployeeNumber IS NOT NULL;
```

Eliminating Duplicates with DISTINCT

Concept

DISTINCT removes duplicates and returns only unique values in the query result.

May Contain Duplicates

```
SELECT country  
FROM customers;
```

Unique Values Only

```
SELECT DISTINCT country  
FROM customers;
```

Business Question

In which distinct countries does our company currently have customers?

Sorting Results with ORDER BY

General Template

```
SELECT column1, column2  
FROM table_name  
ORDER BY column1 ASC, column2 DESC;
```

Ascending (ASC)

```
SELECT productCode, productName, MSRP  
FROM products  
ORDER BY MSRP ASC;
```

Descending (DESC)

```
SELECT productCode, productName, MSRP  
FROM products  
ORDER BY MSRP DESC;
```

Restricting Row Count with LIMIT

Concept

LIMIT restricts the number of rows returned. In MySQL, it is widely used for top-*N* queries and pagination.

First 5 Products

```
SELECT productCode, productName, MSRP
FROM products
LIMIT 5;
```

Top 5 Most Expensive Products

```
SELECT productCode, productName, MSRP
FROM products
ORDER BY MSRP DESC
LIMIT 5;
```

Lexical Order vs. Logical Execution Order

Written (Lexical) Order

- 1 SELECT
- 2 FROM
- 3 WHERE
- 4 ORDER BY
- 5 LIMIT

Logical Execution Order

- 1 Locate the table (**FROM**)
- 2 Filter rows (**WHERE**)
- 3 Project columns (**SELECT**)
- 4 Sort result rows (**ORDER BY**)
- 5 Restrict count (**LIMIT**)

A Complete Example Query

```
SELECT
    productCode AS product_code,
    productName AS product_name,
    productLine AS product_line,
    MSRP AS suggested_price
FROM products
WHERE productLine IN ('Classic Cars', 'Motorcycles')
    AND MSRP BETWEEN 100 AND 200
ORDER BY MSRP DESC
LIMIT 10;
```

Query Objective

Retrieve the top 10 most expensive products in Classic Cars and Motorcycles whose suggested retail price is between 100 and 200.

Common Pitfalls for Beginners

Mistake	Fix
Missing commas between columns	Write <code>SELECT col1, col2, col3</code>
Missing <code>FROM</code> clause	Always define the data source table
Unquoted string literals	Use single quotes: <code>country = 'USA'</code>
Comparing <code>NULL</code> with <code>=</code>	Use <code>IS NULL</code> or <code>IS NOT NULL</code>
Overusing <code>SELECT *</code>	Select only necessary columns

Quick Exercise 1

Write SQL queries for each requirement:

- 1 Retrieve customerNumber, customerName, city, and country from customers.
- 2 Retrieve customers located in France or Germany.
- 3 Retrieve products with an MSRP greater than 150.
- 4 Retrieve the 5 most recent orders based on orderDate.

Quick Exercise 1

Write SQL queries for each requirement:

- 1 Retrieve customerNumber, customerName, city, and country from customers.
- 2 Retrieve customers located in France or Germany.
- 3 Retrieve products with an MSRP greater than 150.
- 4 Retrieve the 5 most recent orders based on orderDate.

Hint for Question 4

```
SELECT orderNumber, orderDate, status
FROM orders
ORDER BY orderDate DESC
LIMIT 5;
```

Quick Exercise 2

Write SQL queries for each requirement:

- 1 Retrieve the distinct list of countries from customers.
- 2 Retrieve products whose name contains the substring 'Ford'.
- 3 Retrieve customers who do not have an assigned sales representative.
- 4 Compute `lineTotal = quantityOrdered * priceEach` from `orderdetails`.

Quick Exercise 2

Write SQL queries for each requirement:

- 1 Retrieve the distinct list of countries from customers.
- 2 Retrieve products whose name contains the substring 'Ford'.
- 3 Retrieve customers who do not have an assigned sales representative.
- 4 Compute $\text{lineTotal} = \text{quantityOrdered} * \text{priceEach}$ from orderdetails.

Hints / Solutions

```
SELECT DISTINCT country FROM customers;  
SELECT * FROM products WHERE productName LIKE '%Ford%';  
SELECT * FROM customers WHERE salesRepEmployeeNumber IS NULL;
```

Summary

- **SELECT** is the fundamental statement for querying data in relational databases.
- **FROM** specifies the source table for the query.
- **WHERE** filters rows based on logical conditions.
- **AS** assigns aliases to columns; expressions in **SELECT** produce computed columns.
- **DISTINCT**, **ORDER BY**, and **LIMIT** clean, sort, and paginate query results.
- When drafting queries, think in business questions: *which columns, from which table, under which conditions?*

- DBMS Tutorial: *Basic SELECT Statement with classicmodels*.

<https://vudmvn.github.io/dbms-course/lectures/MySQL/querying-data/select/select-statement-1/>

- GeeksforGeeks: *SQL SELECT Query*.

<https://www.geeksforgeeks.org/sql/sql-select-query/>